

Designing an aircraft when nobody has the whole picture

What happens when three engineering teams each know their own job, and have to guess about everyone else's.

Every airplane is a compromise between specialists who cannot all be in the same room. This is a small, working experiment in letting them coordinate through published, machine-checkable contracts instead of meetings — and an honest account of what broke when we tried it.

1. Why aircraft design is hard in a specific way

Suppose you want to make an airplane's wings longer and thinner. Long thin wings are efficient — gliders have them for a reason — so this seems like an easy win. But a longer wing bends more, so it needs more structural material, so it gets heavier. A heavier airplane needs more lift, which means a bigger wing, which is heavier still. And a heavier airplane needs a bigger engine, which weighs more too.

You cannot evaluate the change on its own. This is called **coupling**: the pieces of the design determine each other, in loops. Change one number and you have to chase the consequences all the way around and back to where you started.

There is a standard answer. Put every calculation into one computer program, give it to an optimizer, and let it solve all the loops simultaneously. This works, it is well understood, and for a problem the size of ours it would find a better answer faster than anything in this document.

THE CATCH

Real engineering organizations cannot do that. The aerodynamics group runs different software than the structures group, on a different schedule, with different people, sometimes at a different company. Their tools cannot be merged into one

program — not because nobody thought of it, but because the tools, the data, and the expertise genuinely live in separate places.

So what actually happens is this: each team does its own work, using **assumptions** about what the other teams will produce. Aerodynamics assumes the airplane will weigh about 390 kg empty and designs around that. Weights assumes the wing structure will come in around 165 kg. Everyone is working hard and correctly — on slightly different airplanes.

Occasionally they compare notes, discover the assumptions have drifted apart, and redo work. The question this project asks is: **can that comparison be made automatic, precise, and continuous, instead of occasional and verbal?**

2. The idea: publish a region, not a point

The usual thing to hand another team is a design — one airplane, one set of numbers. The problem is that a single design is brittle: change any assumption behind it and it may be worthless, and you cannot tell how close to worthless it was.

Instead, each team here publishes a **region**: the whole set of designs it considers acceptable, described as mathematical formulas over the shared design variables. Six numbers describe an airplane in this experiment — wing area, **aspect ratio** (how long and thin the wing is), thickness, sweep angle, total mass, and engine power. A published region says, in effect, "for any combination of those six numbers, here is how much margin I have."

Four rules make those publications useful.

Everyone must be talking about the same design space

Each publication carries a fingerprint (a **hash**) of the variable list it was built against. If two teams' fingerprints differ — someone added a variable, someone else did not notice — the software refuses to combine them rather than producing a plausible, wrong answer.

Publish margins, not yes/no

"Feasible" throws away the information you need. Every constraint is instead reported as a number scaled so that **0 means exactly at the limit** and -0.10 means "10% of the way to trouble remaining." That scaling makes a structures margin and an aerodynamics margin directly comparable, which is what lets a coordinator rank designs instead of just sorting them.

Say where your model is valid

Prediction models do not fail loudly outside the range they were built for. They keep returning confident, wrong numbers. So each publication declares the box of designs it actually tested, and anything outside is rejected rather than trusted.

Declare what you assumed about everyone else

This is the load-bearing one. Each team records, in machine-readable form, what it assumed about the others — and how far reality may drift from that assumption before its own work must be redone. That declaration is what makes disagreement detectable by a program rather than by a person noticing in a meeting.

3. Three teams, real physics

The experiment uses three simulated teams. Each one runs genuine engineering calculations — not invented formulas — and none of them can see inside the others.

Team	What it computes	How
● Aerodynamics	Lift, drag, stall speed	A vortex lattice solver (AVL) that models the wing as a grid of tiny spinning filaments of air, coupled to a neural network trained on airfoil simulations
● Structures	Wing weight, how much the wing bends	A beam model that adds up the aerodynamic load along the wing and sizes the spar caps to survive it
● Weights	Empty weight of the whole aircraft	A component build-up: a fuselage sized by the volume it must hold, an engine sized by its power, plus systems and landing gear

How do we know the physics is right? The aerodynamics code was checked against a standard test case published decades ago — a specific wing shape with known answers. It reproduced the published lift slope to **0.14%**. The structures model predicts a wing structure of about 167 kg for a mid-range design; the value seeded into the project before any of this was built was 165 kg. Those are independent agreements, and they are the reason to believe anything downstream.

4. The tools behind the numbers

That table is short on detail in a way that matters, because the credibility of everything after this section rests on it. This project began with all three teams publishing *fitted curves* — formulas somebody wrote down that looked like physics and were tuned to be roughly right. Two of the three now publish numbers that real tools computed, and the formulas they publish are fits *to those runs*. Here is what those tools are, and — just as importantly — what each one cannot do alone.

Tool	Gives you	Cannot give you
AVL vortex lattice	How lift is distributed along the wing, drag due to lift, stability	Anything caused by air's stickiness — no skin friction, no stall
NeuralFoil airfoil neural net	Drag and maximum lift of a 2-D wing cross-section	Anything three-dimensional — it does not know a wing exists
Beam sizing structural analysis	How much material the wing needs, and how far it bends	The load it is carrying — that has to come from somewhere

The three gaps are complementary, and closing them costs almost nothing. That is the whole design of this part of the system.

AVL, and the problem of letting software drive it

A **vortex lattice** method divides the wing into a grid of panels, places a spinning filament of air on each, and solves for how strongly each one must spin so that air flows smoothly along the wing surface everywhere. From that you get the lift on every strip of the wing. It is inviscid — air is treated as frictionless — which is why it is fast, and why it is silent

about drag from friction and about stall. AVL is the standard implementation, written at MIT and in use for decades.

The physics was never the hard part. The difficulty is that AVL is an interactive program from the era of text terminals, driven by typing menu commands, and it was written to be operated by a person who can see when something looks wrong. Software driving it blind fails *quietly*. We hit six distinct ways to get a confident, wrong answer, and the wrapper now guards each one:

The trap	The guard
A blank line means “leave this menu”. Writing a results file and then a blank line exits the analysis menu, so the <i>next</i> command is silently ignored and you get a half-empty file with no error.	The command sequence is built deliberately, and a test asserts the exit is where we think it is.
One of the results tables has a heading that contains a space inside what looks like a single column. Reading columns by position therefore returns a neighbouring quantity — plausible, and wrong.	Columns are matched by name , with two tests and a raw-output dump tool for when a number looks odd.
Asking for a plot opens a graphics window and waits forever for someone to close it.	Plot commands are refused before they can reach AVL.
Certain fine panel layouts return “not a number” on some builds.	The geometry generator refuses to build that layout.
Missing geometry, a hung run, or no output at all can all look like a zero result.	Each raises a specific, named error instead of returning a number.

Fourteen automated checks confirm the harness behaves. But internal checks cannot catch a *systematic* error, because the same code writes both the question and the answer. So the tool is also benchmarked against results published outside this project: a standard verification wing whose lift and pitching-moment slopes have been in the literature for decades, and Prandtl’s century-old result that an elliptically loaded wing has an efficiency of exactly 1.0. Ours reproduces the published lift slope to 0.14%.

THE CHECK THAT MATTERS MOST

AVL reports the wing area you told it — comparing that to what you asked for proves nothing. It *also* reports the area it gets by adding up the wing sections it was actually given. Comparing *that* is not circular, and it is what catches a geometry generator

whose trigonometry is subtly wrong — the single most likely error for an automated system to make and never notice.

NeuralFoil, and why not the obvious choice

The standard tool for a 2-D airfoil cross-section is XFOIL. NeuralFoil is a neural network trained on very large numbers of XFOIL runs, and it is the better choice here for a reason specific to this project. XFOIL *fails to converge* on hard cases — thick sections, unusual conditions — and gives you nothing. The aerodynamics team's original declared range of validity was clipped to thin, barely-swept wings for exactly that reason: the tool had refused to answer there.

NeuralFoil does not refuse. It returns an answer together with a **confidence**. So the old justification for that boundary evaporated, and the valid range had to be re-derived from a confidence threshold instead of inherited from a tool limitation that no longer applied. It is also fast enough to evaluate every strip of the wing at once, which is what makes the next part possible.

Bolting them together

One AVL run gives every strip of the wing its own local lift, chord length and area. Each strip therefore has its own flow conditions, and NeuralFoil is asked about each strip separately — one batched call for the whole wing. Two useful things come out.

Friction drag is the area-weighted sum of each strip's own 2-D drag, corrected for wing sweep.

Maximum lift, with a location. Because a vortex lattice is a *linear* solver, every strip's share of the total lift is fixed — increase the wing's lift and each strip's lift scales by the same factor. So the wing stalls exactly when the first strip reaches its own 2-D maximum, and we can compute both when that happens and *where*. That second part is not a detail: a wing that stalls at the root sinks nose-down and recovers, while a wing that stalls at the tip drops a wingtip and rolls. Same number, different airplane.

A FREE RESULT

Because the *shape* of the lift distribution never changes, the strip calculations done at one flight condition can be re-read at any other — no extra solver runs at all. That is what let the aerodynamics team publish a full **drag curve** rather than drag at a

single point. It matters more than it sounds: the previous publication had drag not depending on lift at all, which understated drag-due-to-lift by about **38%** on a typical design. That is not a rounding error on a constraint that decides whether the airplane closes.

The beam model, and one result worth the whole exercise

The structures team's own published limitations used to include the line "span loading held at the seeded distribution" — it was sizing the wing against an assumed load. Since aerodynamics now computes the real one, structures can size against the load the wing actually carries.

The method is classical, and about as far as you can go with pen and paper before you need a full finite-element code. Distribute the lift along the wing for the worst manoeuvre the aircraft must survive; subtract the relief provided by the wing's own weight pushing down; add it up from the tip inward to get the bending moment at every station; then size the load-carrying caps at the top and bottom of the wing box so that each one is working exactly at its allowable stress — a **fully-stressed** design. Mass follows from the volume of material; deflection follows from integrating the curvature.

And then something surprising falls out. If every station is fully stressed, the bending moment cancels out of the deflection calculation entirely. The wing's curvature depends only on the material and the depth of the wing box — not on the load at all.

WHY THIS JUSTIFIES THE WORK

A fully-stressed wing bends **the same amount** whether you design it for a gentle 3g manoeuvre or a violent 6g one. Only its mass changes. That is real, counter-intuitive, and precisely the kind of thing a fitted curve gets silently wrong: the old structures publication had deflection growing in proportion to the load, which for this kind of wing is simply not true. It also means the wing's weight and its stiffness are far less linked than the old formulas implied — which changes the shape of the acceptable region, not just the numbers in it. It is asserted as an automated test.

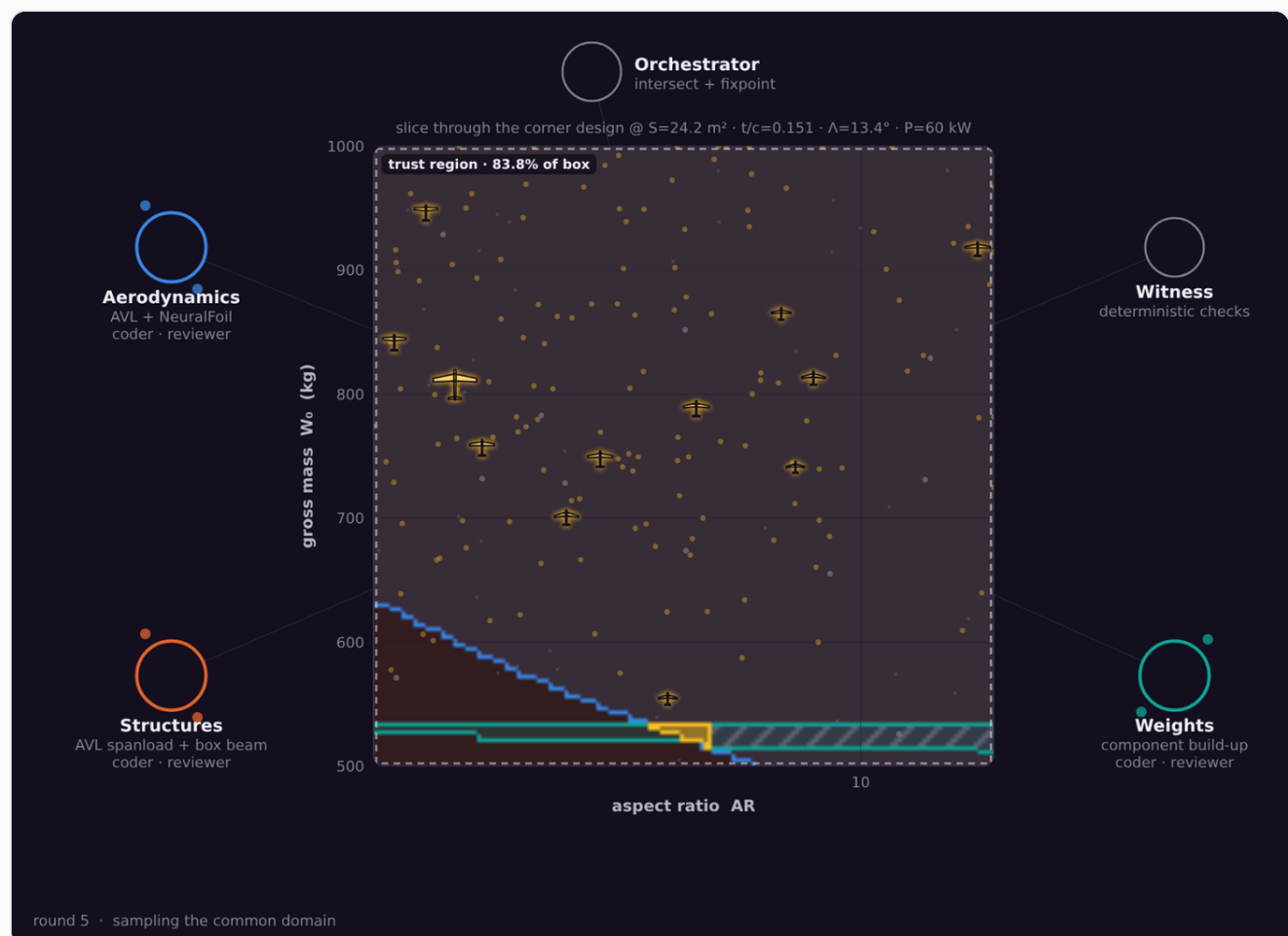
What is still assumed, stated plainly

The AVL model is a *wing*. There is no fuselage, tail, nacelle or landing gear in it, so nothing in the strip calculation can produce their drag. That term is still an estimate from a

standard hand method, and it is written into the published formula as its own separate piece rather than blended in to look measured. It disagrees with the weights team's independent build-up by roughly a quarter to a third — a disagreement we deliberately left visible rather than reconciled away.

The beam has no twisting, no feedback from the wing bending into the airflow, no buckling (which makes it optimistic on the compressed upper cap), no gust cases, no fatigue and no landing loads. The maximum-lift calculation applies a 2-D result strip by strip, so it has no gradual spreading of stall across the wing. All of this is enormously more real than a fitted power law. None of it is a wind tunnel, and the publications say so in their own machine-readable limitations.

5. What the map shows



A two-dimensional slice through the six-dimensional design space: wing slenderness across the bottom, total aircraft mass up the side. Coloured outlines are each team's boundary of acceptability. Gold marks designs acceptable to everyone AND built on assumptions that hold. Diagonal hatching marks the trap: designs every team approves of, where the teams were nonetheless imagining different airplanes. Small dots are candidates the search has evaluated.

That hatched area is the whole point of the project. Those designs pass every individual check. They are still not real, because the checks were performed under assumptions that contradict each other. A coordinator looking only at "does everyone approve?" would report them as successes.

6. What happened over twelve rounds

The teams take turns: publish, compare, update assumptions, repeat. To make it realistic, the experiment starts everyone off **wrong on purpose** and then interrupts them three times with the kind of news that arrives in real projects.

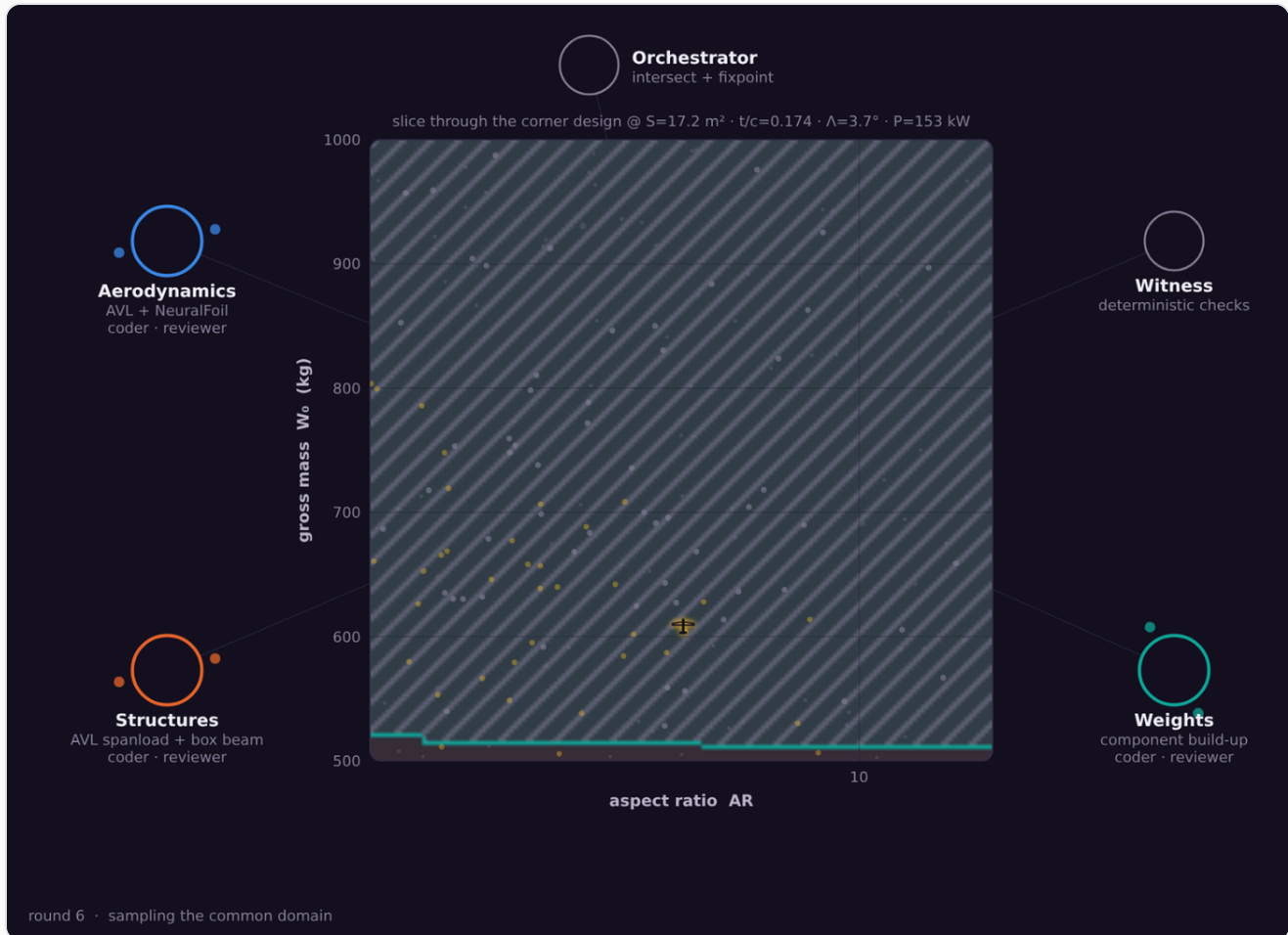
Round	What happened	Usable designs
0	Everyone starts with mistaken beliefs about everyone else	0.0%
3	Structures added the FAR-23 gust load case — the wing must survive a harder load	4.5%
5	Beliefs have caught up; the search is productive	28.3%
6	Structures switched the spar caps to carbon — wings get 53% lighter	0.3%
9	Aerodynamics re-lofted the fuselage and rebuilt the wetted area — less drag	23.1%
11	Recovered	26.4%

Round 6 is the result worth remembering. The structures team switched to carbon fiber spar caps and the wing structure got **53% lighter**. That is unambiguously good news for the airplane. It was catastrophic for the project: every other team was still working from the old, heavier number, so **all 4,000 candidate designs found so far were invalidated at once** and the usable fraction fell to 0.3%.

THE FINDING

An improvement in one place destroyed the accumulated work of everyone else. This is not a bug in the method — it is the method correctly detecting that everybody

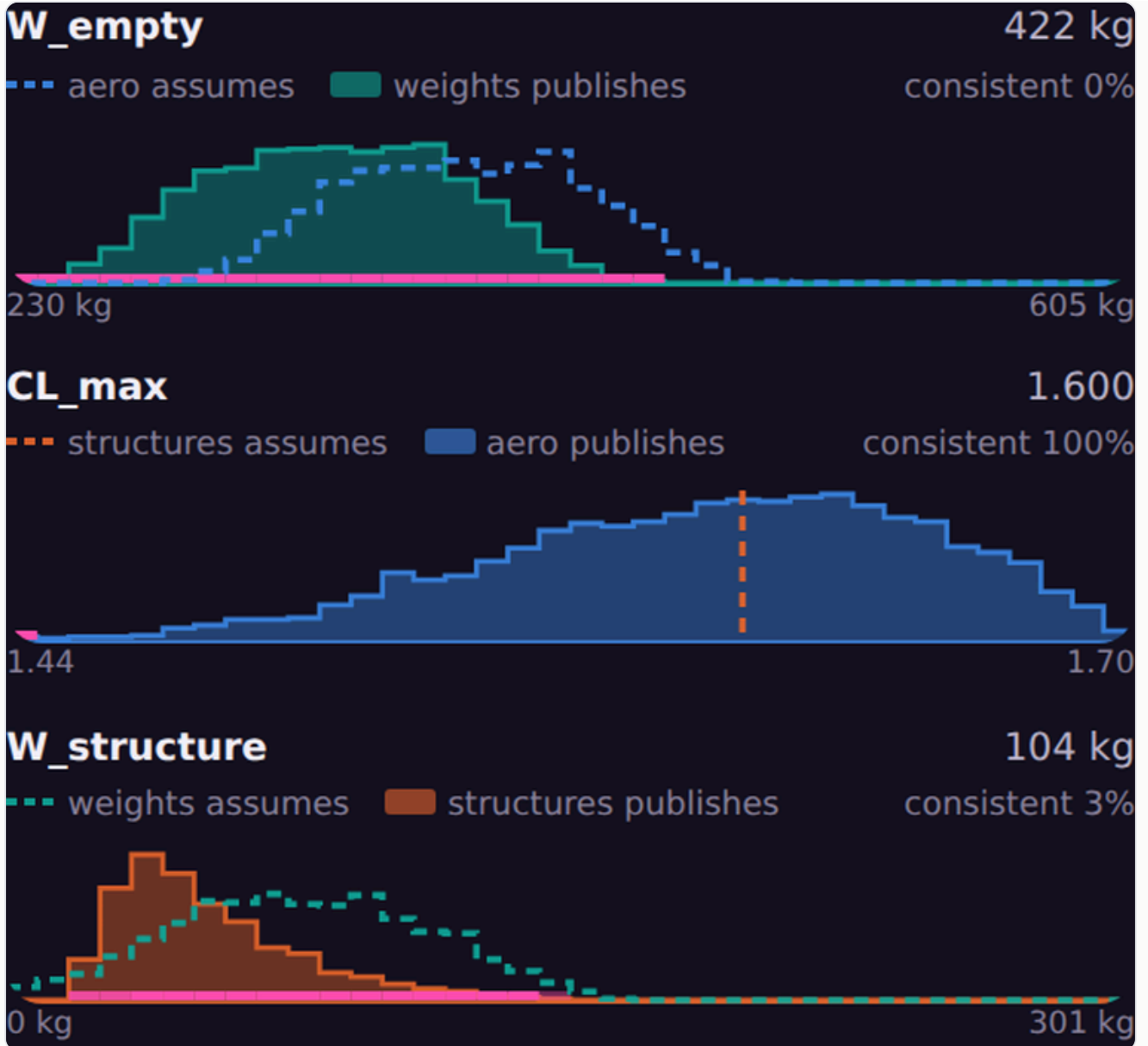
else's homework was based on a number that just changed. In a real project this happens silently and is discovered months later.



The same slice immediately after the carbon change. Everything is hatched: the designs still look acceptable to each team individually, but nobody's assumptions about anybody else are valid any more.

7. Watching teams misunderstand each other

The most useful display we built shows, for each pair of teams, what one *believes* about the other next to what the other *actually publishes*.



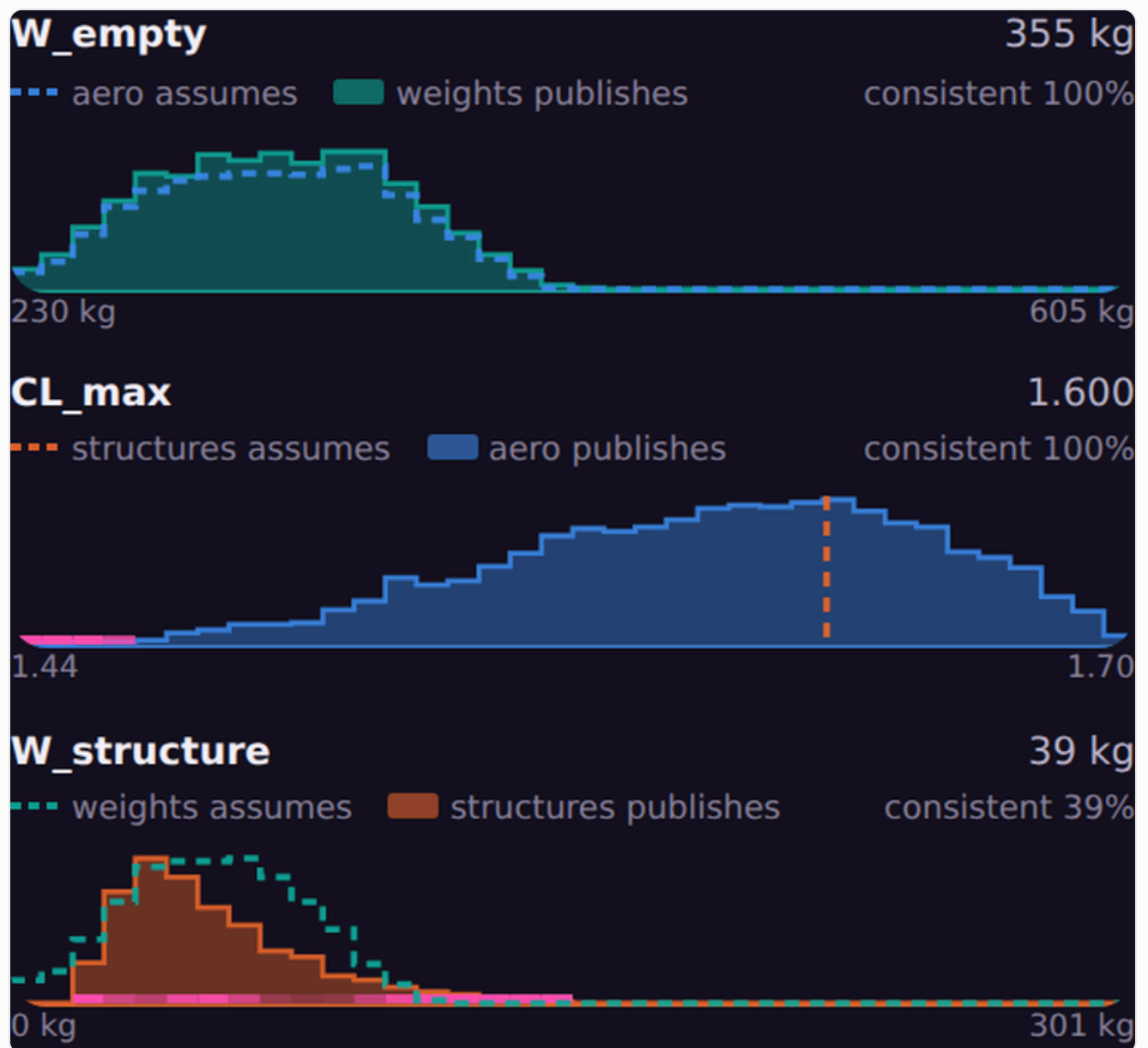
Each panel is one relationship. The filled shape is what a team really publishes across all candidate designs; the dashed outline is what its partner believes. When the two separate, somebody is working from stale information. The red strip along the bottom marks the specific designs where the disagreement exceeds what the consuming team said it could tolerate.

Two details in that picture are worth explaining, because they are easy to misread.

A straight vertical line means a team is holding a single number. In the middle panel, structures assumes one fixed value for the aerodynamic quantity it needs, so its "belief" is a spike rather than a spread. That is a legitimate way to work — it just cannot represent the fact that the real value differs from airplane to airplane.

Two curves can look nearly identical and still disagree badly. The check is performed design by design: for each candidate airplane, does your belief match what your partner actually says *about that airplane*? Overlapping silhouettes hide errors that go one way for

some designs and the other way for others. That is what the red strip is for — it shows *which* designs fail, not just how many.



By the final round the beliefs have largely caught up, and the red strip has shrunk to the regions where the simplified picture one team holds of another is simply too crude to be accurate — a floor set by the model, not by being out of date.

8. Four things we learned by building it

Simple assumptions break when the physics gets real

Originally each team assumed a single number for what its partners would produce. That works if the quantity is roughly constant. Wing structural weight is not: across the design

space it varies by a factor of fourteen. No single number can be within 8% of a quantity that varies fourteen-fold, so almost every design was flagged as inconsistent — and the flag was measuring the crudeness of the assumption rather than any real disagreement.

The fix was to let a team's assumption be a simplified *model* of its partner rather than a single number — the way a real engineer internalizes "structures roughly doubles when the wing gets much longer." Usable designs went from under 1% to about 26%. Nothing about the airplanes changed; only the way disagreement was measured.

Making things precise reveals what you never wrote down

Halfway through, we drew pictures of the candidate airplanes to scale. The fuselage was **too short to attach the tail to** — by up to five meters. The weights team sized the fuselage by the volume it had to hold; the aerodynamics team assumed a tail mounted far behind the wing. Neither was wrong. Nobody owned the relationship between them, so nobody had written it down, so the checking software — which had been reporting everything as consistent — could not see it.

THE UNCOMFORTABLE LESSON

A system that reports "all checks passing" is making a much narrower claim than it appears to. It has verified the relationships somebody thought to declare. The failures that hurt are the ones nobody thought to declare.

The dashboard was the worst offender

The interactive display had its own copy of the physics, written when the project used simple placeholder formulas. It was never updated. For most of this work, every picture anyone looked at was real data drawn on top of year-old physics — airplanes shown as unacceptable that were in fact fine, and tooltips off by a factor of nineteen. The visualization was, in effect, a fourth team working from the stalest assumptions in the building, and it was the only one the contract did not check. It now displays results computed by the engine and has no opinions of its own.

Ask what is actually limiting you before you compromise

Late on, usable designs collapsed to a fraction of a percent, and the obvious response was to relax a requirement — accept a slower airplane, or less range. We measured it instead. Relaxing the range requirement by a third would have improved matters by about

1.7×. Fixing how disagreement was measured improved matters by 6.7×, and cost nothing.

The requirements were never the problem. Without that measurement, the project would have traded away a third of the airplane's endurance to fix a bookkeeping artifact.

9. Why a region instead of “the best airplane”

The obvious objection to all of this: an integrated optimizer would hand you the single best aircraft, and this method never does. That is true, and worth answering directly, because the answer is the strongest argument for the whole approach.

The “best” design is fragile by construction. A constrained optimum is the design with zero slack in several directions at once — that is *why* it is best. It sits exactly where the limits touch. Which makes it the single design most exposed to every piece of future news: a corrected model, a supplier change, a shifted requirement all move the limits, and the design pressed against them moves first and farthest.

WE WATCHED THIS HAPPEN

Each time one of our physics models improved, the computed “best airplane” jumped to a different part of the design space: first a long, slender, thin wing; then a shorter, thicker one; then a compact, moderate one. Three successive model improvements, three different “optimal” aircraft — each held with complete confidence by the tools of its day. Anyone who had committed to the first optimum would have committed to an airplane two revisions of understanding later shown to be wrong. The *region* changed too, but a region degrades gracefully; a point is simply somewhere else.

There is a second mechanism stacked on top. An optimizer does not just tolerate model error — it **seeks it out**, because the place your model is most optimistic looks, to the optimizer, like the best deal on the board. Early-stage models are exactly the kind an optimizer loves to exploit. A region bounded by declared margins and validity limits does not chase the flattering corner.

And early design is when the *requirements themselves* are still moving. Optimizing commits you to one answer for one frozen question. A region is the menu for the

negotiation that actually happens at this stage: when our usable designs collapsed late in the project, the region machinery let us measure that relaxing the endurance requirement would buy a 1.7× improvement while fixing our bookkeeping would buy 6.7× — a trade no single-point answer can even express. The 20 designs in our final shortlist span wingspans from 9.0 to 14.1 meters at essentially the same weight; that spread *is* the trade study, made visible.

To keep the claim honest: a region defers the decision, it does not make it. One airplane eventually gets built, and once requirements freeze and models are trusted, running a focused optimizer *inside* the surviving region is the right final step. The sampling approach also becomes impractical with many more design variables. The argument is not that optimization is wrong — it is that it belongs at the end, and the region is what tells you when the end has arrived.

All of that holds even before the organizational reality: the setting this is built for — separate teams, incompatible tools, updates arriving weeks apart — cannot run an integrated optimizer at all. There, the honest alternative is spreadsheets, email, and finding out at the design review.

Against *that* baseline, look at what the machinery actually produced. Every significant error found during this work was caught by the contract: an aerodynamic correlation being used far outside its valid range, a missing dependence on airflow conditions, a fuselage that could not carry its own tail, a dashboard rendering obsolete physics, and a requirements compromise that would have been made for no reason. None of those discoveries required finding an optimal airplane.

THE CLAIM

The output of this system is not a design. It is **caught errors, a measured picture of how well teams understand each other, and a live map of the trades** that stays useful while models and requirements keep changing. It is closer to automated testing for engineering assumptions than to an optimization method — and in the stage of design where assumptions quietly go stale and requirements are still being argued about, that is the more valuable thing to have.

10. Where the numbers came from

Every figure in this document was produced by the code in this repository, run end to end: three teams publishing from real physics solvers, twelve rounds of coordination, 12,000 candidate designs evaluated per round. The best design found weighs 500 kg; the acceptable region peaked at 14.6% of the design space. The publications carry their own fit quality, validity ranges, and stated limitations, and the checking software refuses to combine publications whose assumptions do not line up.

The things it still cannot do are written down too: no engine model, no gust loads, no buckling analysis, a fixed wing shape, and — most importantly — no way to detect a relationship between teams that nobody declared.

floodlight · generated from aerodynamics-r3-cb659d5e, structures-r2-c44c2a7e, weights-r2-3ba60b4e · 12 rounds · interactive version in <docs/timelapse.html>